

# Arquitectura de Chatgpt

## 0. LLM (LARGE LANGUAGE MODEL)

### Investigar y responder:

Identifica las fuentes de datos masivas utilizadas para entrenar modelos de lenguaje:

#### 1. Corpus web

Son colecciones masivas de información extraída de internet, que incluyen páginas públicas, blogs, foros, enciclopedias digitales y artículos. Este tipo de datos aporta una gran diversidad de temas, estilos de escritura y lenguaje cotidiano, lo que permite a los modelos aprender cómo se utiliza el lenguaje en distintos contextos reales. Sin embargo, requieren procesos de limpieza y filtrado debido a la presencia de información redundante, errores o contenido de baja calidad.

#### 2. Libros digitalizados

Incluyen obras literarias, textos académicos, ensayos y publicaciones formales en formato digital. Estas fuentes se caracterizan por tener una estructura más cuidada, coherente y con un uso más rico del lenguaje. Son fundamentales para que los modelos desarrollen una mejor comprensión gramatical, capacidad de análisis en textos largos y manejo de estilos formales y complejos.

#### 3. Repositorios de código

Abarcan plataformas como GitHub y documentación técnica asociada. Contienen código fuente, comentarios de desarrolladores, manuales y guías técnicas. Estos datos permiten a los modelos aprender lenguajes de programación, estructuras lógicas, sintaxis y patrones de desarrollo, lo que resulta clave para tareas relacionadas con programación y tecnología.

#### 4. Datasets curados

Son conjuntos de datos seleccionados, organizados y filtrados cuidadosamente, ya sea por expertos o mediante procesos automatizados. Su objetivo es garantizar alta calidad, coherencia y relevancia. Pueden incluir datos etiquetados, preguntas y respuestas, diálogos

o información especializada, lo que ayuda a mejorar el rendimiento del modelo en tareas específicas y a reducir errores o sesgos.

Describe el objetivo principal de un modelo de lenguaje:

- modelar la probabilidad de secuencias de texto
- predecir el siguiente token dado un contexto

El objetivo de un LLM es modelar la distribución de probabilidad del lenguaje natural, es decir, aprender qué tan probable es que una secuencia de palabras ocurra.

En términos prácticos:

- Dado un contexto (palabras previas), el modelo predice el siguiente token más probable.
- Esto permite generar texto coherente, responder preguntas y completar ideas.

Explica qué es un token en el contexto de modelos de lenguaje:

- unidad mínima de texto procesada por el modelo
- relación entre texto crudo y tokens

Un token es la unidad básica de procesamiento del modelo. No siempre corresponde a una palabra completa.

Ejemplo:

- “inteligencia” → ["intel", "igencia"]
- “AI” → ["A", "I"]

Esto permite:

- Reducir el tamaño del vocabulario

- Manejar palabras desconocidas mediante subpalabras

Describe el proceso general de funcionamiento de un LLM:

1. El usuario introduce texto.
2. El texto se convierte en tokens.
3. Los tokens pasan por múltiples capas del modelo Transformer.
4. El modelo calcula probabilidades para el siguiente token.
5. Se selecciona un token y se agrega a la salida.
6. El proceso se repite hasta completar la respuesta.

### **1. Entrada del usuario**

El proceso inicia cuando el usuario introduce texto en lenguaje natural (una pregunta, instrucción o comentario). Este texto puede variar en longitud y complejidad, y constituye el contexto inicial que el modelo utilizará para generar una respuesta coherente.

### **2. Tokenización del texto**

El texto ingresado no se procesa directamente como palabras completas, sino que se divide en unidades más pequeñas llamadas *tokens*. Estos pueden ser palabras, subpalabras o incluso caracteres. La tokenización permite al modelo manejar vocabularios amplios y representar el lenguaje de forma más eficiente en términos numéricos.

### **3. Procesamiento en el modelo Transformer**

Los tokens se convierten en vectores numéricos (embeddings) y se introducen en múltiples capas de una arquitectura basada en el concepto de Transformer. Dentro de estas capas, el modelo utiliza mecanismos como la autoatención (*self-attention*) para analizar la relación entre los tokens y comprender el contexto global del texto, permitiendo identificar dependencias incluso a larga distancia.

### **4. Cálculo de probabilidades**

Una vez procesada la información a través de las capas, el modelo genera una distribución de probabilidad sobre todos los posibles tokens que podrían aparecer a continuación. Este

paso se basa en lo aprendido durante el entrenamiento, donde el modelo optimiza su capacidad para predecir el siguiente elemento en una secuencia.

### **5. Selección del siguiente token**

A partir de las probabilidades calculadas, se selecciona un token. Esta selección puede hacerse de distintas formas (por ejemplo, eligiendo el más probable o aplicando técnicas de muestreo para generar respuestas más variadas y naturales). El token elegido se añade a la secuencia de salida.

### **6. Generación iterativa de la respuesta**

El nuevo token generado se incorpora al contexto y el proceso se repite: el modelo vuelve a calcular probabilidades para el siguiente token considerando toda la secuencia acumulada. Este ciclo continúa de manera iterativa hasta que se alcanza una condición de parada, como completar la idea, llegar a un límite de longitud o generar un token especial de finalización.

En conjunto, este proceso permite a un modelo de lenguaje grande (LLM) construir respuestas de forma progresiva, manteniendo coherencia y contexto a lo largo de toda la generación del texto.

Explica la arquitectura base de los LLM modernos:

#### **1. Transformer**

La base de los LLM modernos es la arquitectura Transformer, diseñada para procesar secuencias de datos de manera eficiente y en paralelo. A diferencia de modelos anteriores como las redes recurrentes (RNN), el Transformer no procesa los datos de forma secuencial estricta, sino que utiliza mecanismos de *autoatención* (*self-attention*) para analizar todas las posiciones de la secuencia al mismo tiempo. Esto le permite capturar relaciones entre palabras cercanas y lejanas dentro de un texto, mejorando significativamente la comprensión del contexto.

El Transformer está compuesto por múltiples capas apiladas, donde cada una incluye componentes clave como la autoatención multi-cabeza (*multi-head attention*) y redes neuronales feed-forward. Estas capas transforman progresivamente la representación de los tokens, refinando el significado del texto a medida que avanza el procesamiento.

## 2. modelo decoder-only

Los LLM modernos, como los utilizados en asistentes conversacionales, suelen basarse en una variante llamada modelo *decoder-only* del Transformer. En esta configuración, solo se utiliza la parte decodificadora de la arquitectura original, que está diseñada específicamente para generar texto de manera autoregresiva.

En un modelo *decoder-only*, cada token generado depende únicamente de los tokens anteriores en la secuencia. Esto se logra mediante el uso de una máscara en el mecanismo de atención que impide al modelo “ver” palabras futuras durante el entrenamiento y la generación. Gracias a esta característica, el modelo aprende a predecir el siguiente token paso a paso, lo que lo hace ideal para tareas como generación de texto, chat, traducción y autocompletado.

### **Relación entre ambos conceptos**

En conjunto, el Transformer proporciona la estructura general para entender y procesar lenguaje, mientras que el enfoque *decoder-only* define cómo se utiliza esa estructura para generar texto de forma secuencial. Esta combinación permite a los LLM modernos producir respuestas coherentes, mantener el contexto y adaptarse a diferentes tipos de tareas lingüísticas.

Describe el mecanismo de self-attention:

- Permite al modelo evaluar la importancia de cada token respecto a los demás.

El mecanismo de **self-attention**, componente central de la arquitectura Transformer, permite que un modelo de lenguaje analice una secuencia completa de tokens y determine cómo se relaciona cada uno con los demás dentro del mismo contexto.

### **Evaluación de la importancia entre tokens**

En lugar de procesar cada palabra de forma aislada, el self-attention asigna un peso o nivel de importancia a cada token respecto a todos los demás tokens de la secuencia. Esto significa que, para interpretar una palabra específica, el modelo puede “prestar más atención” a aquellas palabras que aportan mayor significado en ese contexto. Por ejemplo, en una oración, el modelo puede identificar qué términos influyen más en el sentido de una palabra ambigua.

- Captura relaciones contextuales

Gracias a este mecanismo, el modelo logra capturar relaciones tanto cercanas como lejanas dentro del texto. A diferencia de enfoques tradicionales que tienen dificultades para manejar dependencias a larga distancia, el self-attention permite conectar palabras que están separadas por varias posiciones en la oración, manteniendo coherencia semántica y sintáctica.

En conjunto, el self-attention es lo que permite a los LLM comprender el contexto global de una secuencia, interpretar correctamente el significado de las palabras según su entorno y generar respuestas más precisas y coherentes.

Explica el concepto de contexto (context window):

### **Concepto de *context window***

El *context window* (ventana de contexto) se refiere a la cantidad máxima de tokens que un modelo puede procesar simultáneamente dentro de la arquitectura Transformer. Esto

incluye tanto el texto de entrada del usuario como los tokens que el propio modelo va generando durante la respuesta. En términos simples, es “la memoria activa” del modelo en un momento dado.

### **Cantidad de tokens procesables**

Cada modelo tiene un límite específico de tokens que puede manejar a la vez. Si el texto supera ese límite, el modelo no podrá considerar toda la información completa; en algunos casos, los tokens más antiguos se recortan o se descartan. Esto implica que solo una parte del contenido total estará disponible para influir en la generación de la respuesta.

### **Impacto en la calidad de la respuesta**

El tamaño del *context window* tiene un impacto directo en la calidad y coherencia de las respuestas. Una ventana más amplia permite al modelo recordar más información previa, mantener mejor el hilo de una conversación larga y entender documentos extensos con mayor precisión. Por el contrario, una ventana limitada puede hacer que el modelo pierda contexto importante, genere respuestas menos coherentes o repita información, especialmente en interacciones prolongadas o textos complejos.

En conjunto, el *context window* es un factor clave en el rendimiento de los modelos de lenguaje, ya que determina cuánta información pueden utilizar para comprender y generar texto de manera efectiva.

Es el número máximo de tokens que el modelo puede procesar al mismo tiempo.

Impacto:

- Contexto pequeño → respuestas limitadas
- Contexto grande → mejor coherencia y memoria

Describe el proceso de generación de texto:

- predicción probabilística del siguiente token

En los modelos de lenguaje basados en la arquitectura Transformer, la generación de texto se fundamenta en un proceso probabilístico. A partir de una secuencia de tokens de entrada (el contexto), el modelo calcula una distribución de probabilidad sobre todos los posibles tokens que podrían aparecer a continuación. Cada token candidato recibe una probabilidad según qué tan coherente y probable sea en ese contexto, basándose en los patrones aprendidos durante el entrenamiento.

- generación iterativa token por token

Una vez obtenida esta distribución, el modelo selecciona un token (ya sea el más probable o mediante técnicas de muestreo para introducir variabilidad) y lo añade a la secuencia de salida. Este nuevo token pasa a formar parte del contexto, y el proceso se repite: el modelo vuelve a calcular probabilidades considerando ahora la secuencia ampliada.

Este ciclo iterativo continúa generando un token a la vez hasta que se cumple una condición de finalización, como alcanzar un límite de longitud o producir un token especial de fin. Gracias a este mecanismo paso a paso, el modelo construye respuestas completas manteniendo coherencia y fluidez a lo largo del texto.

Explica el concepto de embeddings:

### **Representación vectorial de tokens**

En los modelos de lenguaje basados en la arquitectura Transformer, los *embeddings* son representaciones numéricas (vectores) de los tokens. Cada palabra, subpalabra o símbolo se transforma en un conjunto de valores en un espacio multidimensional. Esta conversión permite que el modelo procese el lenguaje de forma matemática, ya que las redes neuronales operan sobre números y no directamente sobre texto.

### **Captura de relaciones semánticas**

Una de las características más importantes de los *embeddings* es que logran capturar relaciones semánticas entre los tokens. En este espacio vectorial, palabras con significados



similares tienden a ubicarse cerca unas de otras, mientras que palabras con significados distintos se encuentran más alejadas. Por ejemplo, términos relacionados como “médico” y “hospital” tendrán representaciones cercanas, mientras que palabras no relacionadas estarán más distantes.

Gracias a esta propiedad, los modelos pueden entender similitudes, analogías y contextos, lo que mejora su capacidad para interpretar el lenguaje y generar respuestas coherentes.

Describe las limitaciones de los LLM:

### **1. Alucinaciones**

Los modelos de lenguaje pueden generar información que suena coherente y convincente, pero que en realidad es incorrecta o completamente inventada. Esto ocurre porque su objetivo es predecir el texto más probable, no verificar la veracidad de los datos. Como resultado, pueden producir respuestas plausibles sin una base factual sólida, especialmente en temas muy específicos o poco documentados.

### **2. Dependencia del entrenamiento previo**

Un LLM aprende exclusivamente a partir de los datos con los que fue entrenado. Esto implica que su conocimiento está limitado a la información disponible en esos conjuntos de datos y al momento en que se recopilaban. Si un tema no fue incluido o está poco representado durante el entrenamiento, el modelo tendrá dificultades para responder correctamente o simplemente no podrá hacerlo con precisión.

### **3. Falta de acceso a información en tiempo real**

Por sí mismos, los modelos basados en arquitecturas como Transformer no tienen acceso directo a fuentes externas ni a información actualizada. Esto significa que no pueden consultar eventos recientes, cambios en datos o información en tiempo real, a menos que se integren con herramientas adicionales (como navegadores o bases de datos externas). Esta limitación puede afectar la precisión en contextos donde la actualidad es crucial.

Explica por qué los LLM requieren técnicas adicionales para mejorar su desempeño:

### **1. Necesidad de técnicas adicionales**

Aunque los modelos basados en la arquitectura Transformer son muy potentes, su entrenamiento generalista no siempre es suficiente para tareas específicas, contextos especializados o información actualizada. Por ello, se emplean técnicas adicionales que permiten adaptar, guiar y complementar su funcionamiento para mejorar precisión, relevancia y utilidad en aplicaciones reales.

### **2. Fine-tuning (ajuste fino)**

El *fine-tuning* consiste en continuar el entrenamiento del modelo con un conjunto de datos más específico y enfocado en una tarea concreta (por ejemplo, medicina, derecho o atención al cliente). Esto permite que el modelo ajuste sus parámetros internos para especializarse, mejorar su desempeño en ese dominio y reducir errores, manteniendo al mismo tiempo el conocimiento general adquirido previamente.

### **3. Prompt engineering**

El *prompt engineering* se basa en diseñar cuidadosamente la forma en que se le da la instrucción al modelo. La redacción, el contexto proporcionado, los ejemplos incluidos y la claridad del mensaje influyen directamente en la calidad de la respuesta. Al optimizar la entrada, se puede guiar al modelo para que genere resultados más precisos, estructurados y alineados con lo que se necesita, sin modificar su entrenamiento interno.

### **4. Retrieval (RAG: Retrieval-Augmented Generation)**

El enfoque de *retrieval*, como en RAG, combina el modelo con fuentes externas de información (bases de datos, documentos o internet). Antes de generar la respuesta, el sistema recupera información relevante y la incorpora al contexto del modelo. Esto permite acceder a datos actualizados, reducir alucinaciones y mejorar la exactitud en temas específicos o recientes, superando las limitaciones del conocimiento estático del modelo.

## **1. DATA INGESTION / VERSIONING**

## **Investigar y responder:**

Identifica las fuentes de datos masivas utilizadas para entrenar modelos de lenguaje:

### **1. Corpus web**

Son colecciones masivas de información extraída de internet, que incluyen páginas públicas, blogs, foros, enciclopedias digitales y artículos. Este tipo de datos aporta una gran diversidad de temas, estilos de escritura y lenguaje cotidiano, lo que permite a los modelos aprender cómo se utiliza el lenguaje en distintos contextos reales. Sin embargo, requieren procesos de limpieza y filtrado debido a la presencia de información redundante, errores o contenido de baja calidad.

### **2. Libros digitalizados**

Incluyen obras literarias, textos académicos, ensayos y publicaciones formales en formato digital. Estas fuentes se caracterizan por tener una estructura más cuidada, coherente y con un uso más rico del lenguaje. Son fundamentales para que los modelos desarrollen una mejor comprensión gramatical, capacidad de análisis en textos largos y manejo de estilos formales y complejos.

### **3. Repositorios de código**

Abarcan plataformas como GitHub y documentación técnica asociada. Contienen código fuente, comentarios de desarrolladores, manuales y guías técnicas. Estos datos permiten a los modelos aprender lenguajes de programación, estructuras lógicas, sintaxis y patrones de desarrollo, lo que resulta clave para tareas relacionadas con programación y tecnología.

### **4. Datasets curados**

Son conjuntos de datos seleccionados, organizados y filtrados cuidadosamente, ya sea por expertos o mediante procesos automatizados. Su objetivo es garantizar alta calidad, coherencia y relevancia. Pueden incluir datos etiquetados, preguntas y respuestas, diálogos o información especializada, lo que ayuda a mejorar el rendimiento del modelo en tareas específicas y a reducir errores o sesgos.

Describe el proceso de data collection at scale:

### 1. Scraping (recolección automatizada de datos)

El *scraping* consiste en extraer grandes volúmenes de información desde sitios web de forma automatizada. Se utilizan herramientas y scripts para recopilar texto de páginas públicas, foros, blogs y otros recursos accesibles en internet. Este proceso permite construir corpus extensos de manera eficiente, aunque debe realizarse respetando aspectos legales, políticas de uso y privacidad de los datos.

### 2. Datasets públicos

Además del scraping, se emplean conjuntos de datos ya disponibles públicamente, creados por instituciones académicas, organizaciones o comunidades. Estos datasets suelen estar organizados y, en muchos casos, previamente limpiados o anotados, lo que facilita su uso en el entrenamiento. Incluyen desde colecciones de textos hasta datos especializados en áreas como ciencia, medicina o tecnología.

### 3. Filtrado inicial

Una vez recolectados los datos, se realiza un proceso de filtrado inicial para mejorar su calidad. Esto implica eliminar contenido duplicado, irrelevante, ofensivo o incorrecto, así como normalizar formatos y limpiar errores. Este paso es fundamental para reducir ruido en el entrenamiento y asegurar que el modelo aprenda a partir de información más confiable y representativa.

Explica si el entrenamiento de un LLM ocurre bajo:

El entrenamiento de un LLM se realiza **principalmente bajo un enfoque de *batch processing***, aunque puede incorporar elementos de *streaming* en etapas específicas del pipeline de datos.

#### 1. Batch processing (enfoque principal)

En arquitecturas como Transformer, el entrenamiento se lleva a cabo procesando grandes conjuntos de datos divididos en *batches* (lotes) de ejemplos. Cada batch contiene múltiples

secuencias de tokens que se procesan en paralelo utilizando hardware especializado como GPUs o TPUs.

Técnicamente, este enfoque es necesario porque:

- Permite calcular gradientes de forma eficiente mediante operaciones matriciales paralelizables.
- Optimiza el uso de memoria y acelera el entrenamiento al aprovechar el paralelismo.
- Facilita la aplicación de algoritmos de optimización como descenso de gradiente estocástico (SGD) y sus variantes (por ejemplo, Adam).
- Proporciona estabilidad en la convergencia del modelo al promediar el error sobre múltiples ejemplos en cada actualización.

## **2. Streaming (uso complementario)**

El *streaming* no es el método principal de entrenamiento, pero puede utilizarse en la fase de ingestión de datos. En este contexto, los datos se cargan de manera continua desde almacenamiento (disco, red o datasets distribuidos) en lugar de estar completamente en memoria. Esto es útil cuando los volúmenes de datos son extremadamente grandes.

Sin embargo, desde el punto de vista del aprendizaje, incluso si los datos llegan en streaming, el modelo sigue entrenándose en batches, ya que necesita agrupar ejemplos para calcular gradientes de manera eficiente.

## **3. Justificación técnica**

El entrenamiento puramente en streaming (token por token o ejemplo por ejemplo) sería ineficiente y numéricamente inestable en modelos de gran escala. Esto se debe a que:

- No aprovecharía el paralelismo del hardware moderno.
- Generaría actualizaciones de gradiente muy ruidosas.
- Incrementaría significativamente el tiempo de entrenamiento.

Por ello, el *batch processing* es el estándar en el entrenamiento de LLM, mientras que el *streaming* se limita a optimizar el flujo de datos, no el mecanismo central de aprendizaje

Investiga cómo se implementa el versionado de datasets en sistemas de ML:

### **1. Dataset snapshots (instantáneas de datos)**

El versionado de datasets en ML suele comenzar con la creación de *snapshots*, que son “fotografías” completas de un conjunto de datos en un momento específico. Cada snapshot captura el estado exacto del dataset (contenido, estructura y metadatos), permitiendo volver a esa versión en cualquier momento. Técnicamente, estos snapshots pueden implementarse mediante almacenamiento inmutable, sistemas de archivos versionados o herramientas especializadas como DVC o Delta Lake, que registran cambios y evitan sobrescribir datos originales. Esto es clave para auditar qué datos se usaron en cada experimento.

### **2. Data lineage (linaje de datos)**

El *data lineage* describe el ciclo de vida completo de los datos: de dónde provienen, cómo han sido transformados y cómo se utilizan en los modelos. Este seguimiento incluye procesos como limpieza, filtrado, enriquecimiento y particionado en conjuntos de entrenamiento, validación y prueba. Herramientas como Apache Atlas o MLflow permiten registrar estas transformaciones, creando un historial trazable. Esto es fundamental para entender cómo afectan los cambios en los datos al rendimiento del modelo.

### **3. Reproducibilidad**

La reproducibilidad implica que un experimento de ML pueda repetirse obteniendo los mismos resultados. Para lograrlo, no basta con versionar el código; también es necesario versionar los datasets, configuraciones, parámetros y entorno de ejecución. Aquí es donde el versionado de datos (snapshots) y el *data lineage* se integran: permiten reconstruir exactamente qué datos y procesos se utilizaron. Sin estas prácticas, pequeños cambios en los datos pueden generar resultados distintos, dificultando la validación y comparación de modelos.

Explica cómo cambia el dataset a lo largo del tiempo:

### **1. Incorporación de nuevos datos**

Con el tiempo, los datasets se amplían mediante la adición de nueva información proveniente de fuentes recientes (por ejemplo, nuevos textos, interacciones o registros). Esto permite que el modelo se mantenga actualizado, capture cambios en el lenguaje, nuevas tendencias y conocimiento emergente. Técnicamente, esta incorporación puede requerir procesos de reentrenamiento o *fine-tuning* para que el modelo integre correctamente la nueva distribución de datos.

### **2. Eliminación de datos problemáticos**

A medida que se audita el dataset, se identifican y eliminan datos que pueden afectar negativamente el rendimiento o la ética del modelo, como contenido duplicado, sesgado, incorrecto, ofensivo o de baja calidad. También se pueden retirar datos por razones legales o de privacidad. Este proceso mejora la calidad general del dataset, pero puede modificar su distribución y, por lo tanto, el comportamiento del modelo.

### **3. Impacto en el modelo**

Los cambios en el dataset influyen directamente en el desempeño del modelo. La incorporación de nuevos datos puede mejorar la precisión, la capacidad de generalización y la actualidad del conocimiento, mientras que la eliminación de datos problemáticos puede reducir sesgos y errores. Sin embargo, estos cambios también pueden generar efectos no deseados, como *data drift* (cambios en la distribución de los datos) o variaciones en el comportamiento del modelo, lo que hace necesario reevaluar, validar y, en muchos casos, volver a entrenar el modelo para mantener su calidad y confiabilidad.

## **2. DATA VALIDATION**

### **Investigar y responder:**

Identifica los principales problemas en datos de lenguaje natural a gran escala:

- ruido: Datos irrelevantes
- duplicación: Sobreajuste
- contenido tóxico: Contenido ofensivo
- sesgos: Distorsión en las respuestas

### **1. Ruido (datos irrelevantes o de baja calidad)**

En grandes corpus de lenguaje natural es común encontrar información incompleta, mal escrita, fuera de contexto o simplemente irrelevante. Este *ruido* dificulta que el modelo aprenda patrones útiles, ya que introduce señales inconsistentes durante el entrenamiento. Como consecuencia, puede afectar la precisión, generar respuestas incoherentes y reducir la capacidad de generalización.

### **2. Duplicación (riesgo de sobreajuste)**

La presencia de datos repetidos o muy similares en el dataset puede provocar que el modelo memorice información en lugar de aprender patrones generales. Esto incrementa el riesgo de *sobreajuste*, donde el modelo funciona bien con datos conocidos pero pierde capacidad para responder correctamente a casos nuevos. Además, la duplicación puede sesgar la probabilidad de ciertas respuestas, haciéndolas aparecer con mayor frecuencia de lo debido.

### **3. Contenido tóxico (lenguaje ofensivo o inapropiado)**

Los datos provenientes de fuentes abiertas pueden incluir lenguaje ofensivo, discriminatorio o inapropiado. Si no se filtran adecuadamente, el modelo puede aprender y reproducir este tipo de contenido. Esto representa un problema ético y de seguridad, ya que puede generar respuestas dañinas o inaceptables en aplicaciones reales.

### **4. Sesgos (distorsión en las respuestas)**

Los datasets reflejan, en muchos casos, sesgos presentes en la sociedad o en las fuentes de donde se obtienen. Estos sesgos pueden estar relacionados con género, cultura, idioma, ideología u otros factores. Como resultado, el modelo puede producir respuestas parciales, estereotipadas o injustas. Detectar y mitigar estos sesgos es un desafío clave para garantizar sistemas más equitativos y confiables.



Explica técnicas de data filtering:

### **1. Heurísticas (reglas basadas en criterios definidos)**

Las heurísticas son reglas simples y eficientes diseñadas para filtrar datos de forma automática según ciertos patrones. Por ejemplo, se pueden eliminar textos demasiado cortos o excesivamente largos, contenido con caracteres inválidos, duplicados, exceso de símbolos o baja diversidad léxica. También se aplican reglas como detección de idioma o eliminación de páginas con estructuras sospechosas (spam). Aunque son rápidas y escalables, su principal limitación es que pueden ser demasiado rígidas y no capturar matices complejos del lenguaje.

### **2. Clasificadores de calidad**

Son modelos de aprendizaje automático entrenados para evaluar la calidad de los datos. Estos clasificadores analizan aspectos como coherencia, relevancia, nivel de toxicidad o valor informativo del texto. A diferencia de las heurísticas, pueden capturar patrones más complejos y adaptarse mejor a diferentes tipos de contenido. Sin embargo, requieren datasets etiquetados para su entrenamiento y pueden introducir sesgos si no se diseñan cuidadosamente.

### **3. Blacklist / Whitelist**

Estas técnicas consisten en listas predefinidas de elementos permitidos o prohibidos.

- **Blacklist:** bloquea palabras, dominios, expresiones o tipos de contenido considerados inapropiados (por ejemplo, lenguaje ofensivo o sitios de baja calidad).
- **Whitelist:** permite únicamente contenido proveniente de fuentes confiables o previamente validadas.

Ambos enfoques son útiles para controlar la calidad y seguridad de los datos, pero pueden ser limitantes: las *blacklists* pueden no cubrir todas las variantes problemáticas, y las *whitelists* pueden restringir demasiado la diversidad del dataset.

Describe cómo se detecta:

### **1. Detección de *data corruption***

La *corrupción de datos* se refiere a errores o alteraciones que vuelven los datos incompletos, ilegibles o inconsistentes. Para detectarla, se emplean validaciones técnicas como verificación de integridad (checksums o hashes), detección de valores nulos o fuera de rango, errores de codificación (por ejemplo, problemas de UTF-8) y análisis de formatos inválidos. También se comparan versiones del dataset para identificar cambios inesperados. Estas técnicas permiten asegurar que los datos no hayan sido dañados durante su almacenamiento o transferencia.

### **2. Detección de anomalías**

Las anomalías son datos que se desvían significativamente del comportamiento esperado. Se identifican mediante métodos estadísticos (como detección de outliers usando desviación estándar o percentiles) o modelos de aprendizaje automático diseñados para detectar patrones inusuales. En datos de lenguaje, esto puede incluir textos extremadamente largos, repeticiones excesivas, contenido incoherente o estructuras poco comunes. Detectar estas anomalías ayuda a evitar que el modelo aprenda patrones incorrectos o poco representativos.

### **3. Detección de distribución no deseada**

Este problema ocurre cuando la distribución de los datos no refleja adecuadamente el objetivo del modelo (por ejemplo, exceso de un tipo de contenido o falta de diversidad). Para identificarlo, se utilizan análisis estadísticos y comparaciones entre conjuntos de datos (entrenamiento, validación y prueba). Métricas como frecuencia de palabras, balance de clases o diversidad temática permiten evaluar si el dataset está sesgado o desbalanceado. También se emplean técnicas de monitoreo para detectar *data drift*, es decir, cambios en la distribución de los datos a lo largo del tiempo.

Investiga métricas o criterios de calidad de datos:

## **1. Diversidad**

La diversidad mide qué tan variado es el contenido dentro del dataset. En lenguaje natural, implica incluir múltiples temas, estilos de escritura, dominios (científico, informal, técnico), idiomas o variantes lingüísticas. Técnicamente, se puede evaluar mediante métricas como riqueza léxica (variedad de vocabulario), distribución de temas (*topic modeling*) o entropía de la distribución de palabras. Una alta diversidad ayuda a que el modelo generalice mejor y no se limite a un tipo específico de contenido.

## **2. Cobertura**

La cobertura se refiere a qué tanto del “espacio de conocimiento” o de posibles casos está representado en el dataset. Un dataset con buena cobertura incluye ejemplos suficientes de los distintos escenarios que el modelo deberá enfrentar. Esto puede medirse analizando la presencia de categorías relevantes, frecuencia de conceptos clave o distribución de ejemplos por clase en tareas supervisadas. Una cobertura insuficiente provoca que el modelo falle en casos poco representados o desconocidos.

## **3. Representatividad**

La representatividad evalúa si el dataset refleja adecuadamente la realidad o el dominio objetivo donde se aplicará el modelo. Esto implica que la distribución de los datos (por ejemplo, temas, estilos, demografía lingüística) sea coherente con el uso esperado. Se analiza comparando el dataset con datos reales mediante estadísticas descriptivas o pruebas de similitud de distribuciones. Una baja representatividad puede introducir sesgos y generar respuestas distorsionadas o poco precisas en contextos reales.

En conjunto, estas métricas permiten evaluar la calidad de los datos desde diferentes perspectivas, asegurando que el modelo no solo aprenda correctamente, sino que también sea robusto, equilibrado y útil en aplicaciones prácticas.

## **3. DATA PREPROCESSING**

## Investigar y responder:

Explica el proceso de tokenización:

- **BPE (Byte Pair Encoding):** es un algoritmo que: Empieza con caracteres individuales y va fusionando los pares de caracteres más frecuentes.
  - **WordPiece:** es similar a BPE, pero no se basa solo en frecuencia, sino en probabilidad, busca maximizar la probabilidad del lenguaje. En lugar de elegir el par más frecuente: Selecciona el par que mejor mejora la probabilidad del modelo, prioriza tokens que hacen al modelo más preciso.
  - **SentencePiece:** es un enfoque más avanzado que:
    - Trata el texto como una secuencia continua de caracteres
    - No depende de espacios, no asume separación por palabras, funciona bien en idiomas sin espacios (como japonés o chino), puede implementar:
      - BPE
      - Unigram Language Model
1. Toma texto crudo (sin dividir)
  2. Aprende directamente:
    - subpalabras
    - tokens
  3. Genera un vocabulario optimizado

Describe cómo se construye un vocabulario tokenizado

Explica el concepto de:

- secuencias de entrada
- longitud de contexto (context window)

Investiga cómo se transforman textos en:

- tokens
- embeddings iniciales

Describe el pipeline de transformación:

- raw text → cleaned text → tokens → input tensors

## 4. MODEL TRAINING

**Investigar y responder:**

Identifica la arquitectura base de ChatGPT:

- Transformer: La arquitectura fundamental de ChatGPT es el **Transformer**, un modelo introducido en 2017 que revolucionó el procesamiento de lenguaje natural.

### **Características principales**

- Procesa secuencias completas en paralelo (no secuencial como RNN o LSTM).
- Se basa en mecanismos de atención en lugar de memoria recurrente.
- Permite capturar relaciones entre palabras aunque estén muy separadas en el texto.

### **¿Por qué es importante?**

Antes del Transformer:

- Los modelos tenían dificultades con dependencias largas.

Con Transformer:

- Se pueden modelar relaciones complejas en grandes textos.
- Escala mucho mejor con grandes volúmenes de datos.
- decoder-only: Solo usa la parte “decoder” del Transformer y está diseñado específicamente para generar texto.

## Funcionamiento

- Recibe una secuencia de entrada
- Predice el siguiente token
- Repite el proceso hasta completar la respuesta

## Diferencia con encoder-decoder

### 1. Modelo Encoder-Decoder

El modelo *encoder-decoder*, basado en la arquitectura Transformer, está diseñado para tareas donde se necesita **transformar una secuencia de entrada en otra secuencia de salida**. El *encoder* procesa el texto de entrada y genera una representación contextual completa, mientras que el *decoder* utiliza esa representación para producir la salida.

Este enfoque es ideal para tareas como traducción automática, resumen de texto o conversión de formatos, donde existe una correspondencia clara entre entrada y salida. Por ejemplo, en traducción, el encoder comprende el texto en un idioma y el decoder genera su equivalente en otro.

### 2. Modelo Decoder-only

El modelo *decoder-only* utiliza únicamente la parte decodificadora del Transformer y está diseñado para **generar texto de forma autoregresiva**, es decir, prediciendo un token a la vez en función de los tokens anteriores.

En este caso, no hay una separación explícita entre comprensión y generación; todo el contexto (incluyendo la entrada del usuario) se trata como una secuencia continua. El modelo aprende a continuar el texto de manera coherente, lo que lo hace especialmente útil para tareas como chat, autocompletado, generación de contenido y asistencia conversacional.

### 3. Diferencia clave

La diferencia principal radica en su funcionamiento:

- El *encoder-decoder* transforma una secuencia en otra (entrada → salida).

- El *decoder-only* extiende una secuencia existente (continuación del texto).

Explica el mecanismo de:

- **self-attention:** es el mecanismo que permite que cada palabra en una secuencia:  
Evalúe su relación con todas las demás palabras

Cada token se transforma en tres vectores:

- Query (Q)
- Key (K)
- Value (V)

El modelo:

1. Compara cada Query con todas las Keys
2. Calcula pesos de relevancia
3. Aplica esos pesos a los Values

- **multi-head attention:** Es una extensión del self-attention donde el modelo usa múltiples “cabezas” de atención en paralelo.
- Se ejecutan varias atenciones simultáneamente
- Cada una aprende diferentes relaciones:
  - Sintácticas (gramática)
  - Semánticas (significado)
  - Posicionales

Luego:

Se combinan los resultados

## **Ventaja**

- Permite analizar el texto desde múltiples perspectivas
- Mejora la capacidad de representación

Describe el objetivo de entrenamiento:

- next-token prediction
- language modeling

Investiga:

- función de pérdida (cross-entropy)
- optimización (Adam, AdamW)

Explica el uso de:

- GPUs / TPUs
- entrenamiento distribuido
- paralelismo de datos y modelo

## **5. MODEL TUNING (Ajuste Fino)**

Tras el pre-entrenamiento masivo, el modelo solo sabe autocompletar texto (es una "enciclopedia caótica"). Para ser un asistente de IA, pasa por:

### **Fine-Tuning Supervisado (SFT)**

El objetivo de esta fase es enseñarle al modelo a **seguir instrucciones** y a adoptar un formato de diálogo o pregunta-respuesta.

- **Datasets etiquetados:** A diferencia del pre-entrenamiento (donde lee toda la web de forma no supervisada), aquí se utilizan conjuntos de datos creados por humanos expertos. Estos datasets consisten en pares perfectos de **[Prompt del**



Usuario] + [Respuesta Ideal].

- **Ajuste de pesos:** El modelo genera una respuesta a un prompt del dataset. Luego, se calcula el error (la diferencia entre su respuesta y la "Respuesta Ideal" humana). Mediante el algoritmo de *backpropagation* (retropropagación), se **ajustan los pesos** matemáticos de sus redes neuronales para que, en el futuro, su respuesta se parezca más a la del ejemplo humano.

## RLHF (Reinforcement Learning with Human Feedback)

El SFT es costoso porque requiere humanos escribiendo respuestas largas. Para escalar el aprendizaje y pulir la calidad, OpenAI utiliza RLHF, que es el verdadero "ingrediente secreto" de ChatGPT.

Se divide en dos grandes componentes:

- **Reward Model (Modelo de Recompensa):** Se toma el modelo y se le hace generar 4 o 5 respuestas diferentes para un mismo prompt. Un humano lee las respuestas y las clasifica de mejor a peor (ej. la respuesta C es mejor que la A). Con estos datos, **se entrena un segundo modelo más pequeño** (el Reward Model) cuya única función es actuar como un "juez virtual". Aprende a calificar textos exactamente como lo haría un humano.
- **Policy Optimization (Optimización de Políticas):** Aquí entra el aprendizaje por refuerzo puro, típicamente usando un algoritmo llamado **PPO (Proximal Policy Optimization)**. El LLM principal (la "política") genera una respuesta, el *Reward Model* la califica, y el LLM actualiza sus pesos para maximizar esa calificación en el futuro. PPO asegura que el modelo mejore sin "destruir" lo que ya aprendió en las fases anteriores (evitando que cambie demasiado rápido).

## El Impacto: Alignment y Control de Comportamiento

- **Alignment (Alineación):** Es el resultado directo del RLHF. Significa alinear los objetivos matemáticos del modelo con los **valores y expectativas humanas**. Un modelo alineado es "HHH": *Helpful* (Útil), *Honest* (Honesto/Veraz) y *Harmless*

(Inofensivo).

- **Control del comportamiento:** Esta fase es lo que permite establecer "barreras de seguridad" (guardrails). A través de castigos en el *Reward Model*, el LLM aprende a rechazar peticiones dañinas. Por ejemplo, si se le pide "cómo hackear un banco", el modelo aprende a generar una respuesta de rechazo educada en lugar de completar la tarea.

## Diferencias Clave: Pretraining vs Fine-tuning vs Alignment

Para el equipo, esta distinción es la base de todo el ciclo de vida del modelo:

- **Pretraining (Pre-entrenamiento):** \* *Objetivo:* Aprender gramática, hechos del mundo y razonamiento básico.
  - *Método:* Predecir la siguiente palabra leyendo terabytes de datos de internet (Autosupervisado).
  - *Resultado:* Un "Modelo Base" (Base Model) que sabe mucho, pero es incontrolable y no sabe conversar.
- **Fine-Tuning (Ajuste Fino - SFT):**
  - *Objetivo:* Aprender el formato de instrucción y especializarse en tareas concretas (ej. resumir, traducir, programar).
  - *Método:* Imitar ejemplos perfectos creados por humanos (Supervisado).
  - *Resultado:* Un "Modelo de Instrucciones" (Instruct Model) que obedece órdenes, pero aún puede generar respuestas sesgadas o peligrosas.
- **Alignment (Alineación - RLHF):**
  - *Objetivo:* Ser seguro, útil y preferible para los humanos.
  - *Método:* Optimización basada en recompensas usando un juez de IA entrenado por humanos (Aprendizaje por Refuerzo).
  - *Resultado:* Un "Modelo de Chat" (Chat Model) como ChatGPT, listo para interactuar con millones de personas en producción.

## 6. MODEL VALIDATION

Se evalúa mediante métricas cuantitativas como la **Perplejidad (Perplexity)** (menor perplejidad = el modelo está menos "sorprendido" por el texto real y, por ende, es mejor) y la precisión a nivel de token. Cualitativamente, se mide la coherencia y factualidad con benchmarks estándar en la industria como **MMLU** o **HumanEval**.

## **Evaluación Cualitativa: Más allá de los números**

Mientras que la "Perplejidad" mide qué tan bien el modelo predice la siguiente palabra, la evaluación cualitativa mide la **utilidad real**.

- **Coherencia:** Evalúa si el texto tiene sentido lógico de principio a fin. Se busca que no haya contradicciones internas (ej. que el modelo no diga "blanco" al inicio y "negro" al final) y que la estructura gramatical sea impecable.
- **Relevancia:** Mide qué tan bien la respuesta se ajusta a la intención del usuario. Un modelo puede ser coherente pero irrelevante si responde algo que no se le preguntó. Se evalúa si el "tono" y la "longitud" son adecuados para el prompt.
- **Factualidad:** Es la veracidad. Se compara la respuesta del modelo con fuentes de verdad (bases de datos, Wikipedia, artículos científicos). Es la métrica más difícil de automatizar y suele requerir expertos humanos o sistemas de búsqueda externa (RAG) para validarla.

## **Evaluación de Riesgos: Hallucinations, Bias y Seguridad**

Estos son los "puntos de fallo" críticos que pueden destruir la confianza en un sistema de IA.

- **Alucinaciones (Hallucinations):** Se evalúan mediante:
  - **Self-Consistency:** Se le pide al modelo la misma respuesta varias veces; si las respuestas varían mucho, hay alta probabilidad de alucinación.
  - **NLI (Natural Language Inference):** Un segundo modelo verifica si la afirmación de la IA está respaldada por un texto de referencia.
- **Sesgos (Bias):** Se evalúan usando datasets de "prompts de prueba" diseñados para forzar estereotipos de género, raza o religión. Se mide si el modelo favorece

sistemáticamente a un grupo sobre otro.

- **Seguridad (Safety):** Se evalúa mediante el **Red Teaming** (un equipo de personas intentando "romper" el modelo). Se prueban:
  - *Jailbreaks*: Intentar que la IA ignore sus reglas de seguridad.
  - *Inyección de Prompts*: Intentar que revele datos privados o instrucciones maliciosas.
  - *Leakage de PII*: Verificar que no revele información personal identificable.

| Métrica                  | Propósito                                                                                     | Interpretación                                                                                           |
|--------------------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Perplexity (Perplejidad) | Mide qué tan bien el modelo predice una muestra de texto. <sup>29</sup>                       | Un valor menor indica que el modelo está menos "sorprendido" por el texto real. <sup>31</sup>            |
| Accuracy (Token-level)   | Mide el porcentaje de tokens predichos correctamente. <sup>31</sup>                           | Útil para tareas de opción múltiple o compleción exacta.                                                 |
| BLEU / ROUGE             | Evalúan la similitud entre el texto generado y una referencia humana. <sup>31</sup>           | Comúnmente usados en traducción y resumen, basados en el solapamiento de n-gramas. <sup>30</sup>         |
| Benchmarks (MMLU, GSM8K) | Pruebas estandarizadas sobre conocimientos generales y razonamiento matemático. <sup>29</sup> | MMLU cubre 57 materias; GSM8K evalúa resolución de problemas matemáticos de nivel escolar. <sup>29</sup> |

## 7. MODEL DEPLOYMENT

En producción, los LLM se sirven mediante APIs REST en infraestructuras de nube. Su escalabilidad técnica requiere mecanismos de ingeniería complejos:

- **KV Caching (Key-Value Caching):** Almacena en memoria las representaciones (claves y valores) de los tokens ya procesados. *Justificación técnica:* Evita recalcular matemáticamente todo el contexto desde cero por cada palabra nueva generada, lo

cual reduce la latencia de forma drástica.

- **Dynamic Batching:** Agrupa peticiones de distintos usuarios en lotes sobre la marcha. *Justificación técnica:* Maximiza el rendimiento (throughput) de las GPUs costosas, procesando múltiples consultas de usuarios diferentes en un solo ciclo de cálculo.

## Servir un LLM en Producción

- **API REST:** Es el estándar de comunicación. El modelo se "envuelve" en un servicio web (usando frameworks como FastAPI o Flask). El cliente envía un prompt vía una petición **POST** con un cuerpo JSON y recibe la respuesta. Esto permite que cualquier aplicación (web, móvil) consuma la IA de forma sencilla.
- **Inferencia en Cloud:** Los LLM requieren hardware especializado (GPUs NVIDIA A100/H100 o TPUs de Google). Servirlos en la nube permite **escalabilidad elástica**: si hay muchos usuarios, el sistema activa más servidores automáticamente; si no hay nadie, los apaga para ahorrar costos.

## Arquitectura Típica

El flujo de información sigue este camino: **Frontend** (Interfaz de usuario) → **API Gateway** (Seguridad, balanceo de carga) → **Backend/Inferencia** (El modelo en GPU) → **Post-procesamiento** (Filtros de seguridad) → **Respuesta**.

## Técnicas Avanzadas de Optimización

Para que el sistema no sea extremadamente lento ni costoso, se usan estas estrategias:

- **Inferencia Distribuida:** Como modelos como GPT-4 son demasiado grandes para una sola tarjeta de video, se dividen entre varias GPUs (Paralelismo de Modelo). Una parte del modelo procesa una fracción de la información y la pasa a la siguiente GPU.
- **Batching de Requests (Dynamic Batching):** Es la técnica más importante. En lugar de procesar la pregunta de "Usuario A" y luego la de "Usuario B", el servidor

espera milisegundos para agrupar 16 o 32 preguntas y las procesa **todas al mismo tiempo** en un solo ciclo de la GPU. Esto multiplica la eficiencia por 10.

## Métricas de Rendimiento

- **Latencia:** Es el tiempo que tarda el usuario en ver el texto. En LLMs se divide en:
  - *Time to First Token (TTFT):* Cuánto tarda en aparecer la primera palabra (sensación de rapidez).
  - *Tokens Per Second (TPS):* La velocidad a la que "escribe" el resto de la respuesta.
- **Throughput:** Es la capacidad total del sistema. Se mide en cuántas consultas (o cuántos miles de tokens) puede procesar el servidor por segundo en total.
  - *Dilema técnico:* A mayor batching, mejor **throughput** (más barato), pero mayor **latencia** (el usuario espera más).

## Gestión y Estabilidad

- **Versionado de Modelos:** Al igual que el software, los modelos tienen versiones (v1.0, v1.1). Se guardan no solo el código, sino los "pesos" (weights) y la configuración. Se usan registros de modelos (como MLflow o Hugging Face Hub) para saber exactamente qué versión está activa.
- **Rollback:** Si después de actualizar a un nuevo modelo detectamos que empieza a "alucinar" más de lo normal o que es ofensivo, el sistema debe ser capaz de hacer un **rollback** inmediato, que consiste en redirigir el tráfico de la API hacia la versión anterior estable en cuestión de segundos.

## 8. MODEL ANALYSIS

El monitoreo continuo es esencial para detectar el **Model Drift** (cuando el modelo pierde eficacia porque el mundo o el lenguaje cambiaron) y el **Data Drift** (cambio en la distribución estadística de las entradas del usuario). Se requieren dashboards de observabilidad y trazas completas de las inferencias.

### ¿Cómo monitorear un LLM en producción?

El monitoreo se sostiene sobre tres pilares fundamentales:

- **Logs (Registros):** A diferencia de una base de datos tradicional, aquí se debe registrar tanto el *prompt* (la entrada del usuario) como la *respuesta* generada. Esto es vital para auditorías posteriores y para entender el uso real del sistema.
- **Métricas:** Se monitorean métricas de ingeniería clásica (uso de CPU/GPU, consumo de memoria) y métricas específicas de LLMs, como la latencia (Time to First Token) y el *throughput* (Tokens por segundo).
- **Trazabilidad (Tracing):** Se requiere almacenar trazas completas de las inferencias. Esto significa rastrear el ciclo de vida completo de la petición: desde que el usuario hace la pregunta, cómo se tokeniza, si el sistema hizo una llamada a una base de datos externa (RAG) y cuánto tiempo tomó la generación final.

## Identificación de Fallas Comunes

Un LLM puede fallar silenciosamente (el sistema no se cae, pero devuelve basura). Las fallas más comunes son:

- **Hallucinations (Alucinaciones):** Ocurre cuando el modelo genera información que parece plausible y segura de sí misma, pero que es factualmente falsa.
- **Respuestas incorrectas:** Fallas en el razonamiento lógico, errores matemáticos o incapacidad para seguir las instrucciones (el *prompt*) que dio el usuario.
- **Degradación del rendimiento:** Ocurre cuando el sistema se vuelve lento (aumento de latencia) debido a picos de tráfico mal gestionados, o cuando los usuarios notan que la calidad de las respuestas ha disminuido (el modelo se vuelve "perezoso").

## ¿Cómo detectar anomalías en el tiempo?

Incluso si el código del modelo no cambia, su rendimiento puede deteriorarse. El monitoreo continuo es esencial para detectar los siguientes fenómenos:

- **Data Drift (Desviación de Datos):** Es un cambio en la distribución estadística de las entradas del usuario. Por ejemplo, el modelo fue entrenado principalmente para responder en inglés, pero de repente el 80% de los usuarios comienza a hacerle

preguntas en español con modismos locales. El modelo empezará a fallar porque la realidad de los datos de entrada cambió.

- **Model Drift (Desviación del Modelo):** Ocurre cuando el modelo pierde eficacia porque el mundo o el lenguaje cambiaron. Por ejemplo, un modelo entrenado hasta 2021 sufrirá de *model drift* si se le pregunta por el último ganador del Mundial, ya que su conocimiento interno se ha vuelto obsoleto.

## Técnicas de Observabilidad

Para gestionar lo anterior a gran escala, la arquitectura requiere:

- **Dashboards (Paneles de control):** Interfaces visuales para observar en tiempo real métricas críticas como el volumen de tráfico, la latencia promedio, la tasa de errores y, si es posible, puntuaciones automatizadas de calidad de respuesta.
- **Alertas:** Notificaciones automáticas (vía Slack, correo o PagerDuty) configuradas para dispararse cuando se rompen ciertos umbrales. Por ejemplo, una alerta si el tiempo de respuesta supera los 5 segundos, o si un sistema automatizado detecta un aumento repentino en el uso de lenguaje tóxico generado por el modelo.

## 9. MODEL FEEDBACK Y EL "FLYWHEEL" DE DATOS

El sistema en producción actúa como un organismo que mejora iterativamente:

- **El Flywheel (Volante de inercia):** Las interacciones de los usuarios (likes, dislikes, ediciones de respuestas) se recolectan constantemente de la aplicación.
- **Reentrenamiento:** Este feedback se etiqueta sistemáticamente y se convierte en conjuntos de datos negativos o positivos para las siguientes rondas de ajuste por RLHF.
- **Justificación Arquitectónica:** Es la única manera comprobada de mantener la relevancia y seguridad del modelo a gran escala, corrigiendo de manera constante los sesgos y alucinaciones descubiertos en la interacción con el mundo real, y estableciendo así un ciclo de aprendizaje continuo robusto.



## ¿Cómo se recolecta el feedback?

OpenAI y otros desarrolladores de LLMs utilizan tres canales principales en la interfaz de usuario:

- **Interacciones:** El sistema analiza de forma agregada cómo interactúan los usuarios. Por ejemplo, si un usuario pide un código, lo copia y no vuelve a preguntar, es una señal de éxito. Si el usuario regenera la respuesta 5 veces, es una señal de que el modelo está fallando.
- **Ratings (Calificaciones):** Son los iconos de **pulgar arriba** (👍) y **pulgar abajo** (👎). Es la señal más clara y directa. Un "pulgar abajo" suele abrir un cuadro de diálogo donde el usuario puede especificar si la respuesta fue "falsa", "irrelevante" o "dañina".
- **Correcciones:** Ocurre cuando el usuario escribe frases como *"No, eso es incorrecto, la capital es..."* o cuando utiliza la función de "Editar" su propio prompt para guiar mejor al modelo. Estas correcciones son "oro puro" para los ingenieros de datos.

## Conversión del Feedback en Mejora

El feedback no entra directamente al cerebro de la IA; primero debe ser procesado:

- **Nuevos Datasets:** Las interacciones marcadas como "buenas" o "malas" se limpian y anonimizan. Con ellas se crean nuevos datasets para el **Reward Model** (el modelo que califica qué es bueno y qué es malo).
- **Reentrenamiento:** Periódicamente, OpenAI toma estos nuevos datasets y ejecuta una nueva ronda de **RLHF** o **SFT**. Esto permite que el modelo "aprenda" de los errores reportados por millones de personas en la versión anterior.

## El Ciclo de Vida del Modelo

Este es el "motor" de la mejora continua en la arquitectura de ChatGPT:

1. **Producción:** El modelo (ej. GPT-4) está disponible para el público.
2. **Monitoreo:** Se observan fallas, latencia y uso de los usuarios (Etapa 8).

3. **Análisis:** Los ingenieros identifican patrones (ej. "el modelo está siendo muy rudo en español").
4. **Mejora:** Se recolecta feedback específico y se crean nuevos datos correctivos.
5. **Nuevo Modelo:** Se entrena una versión mejorada (ej. GPT-4o) y se despliega, reiniciando el ciclo.

## Aprendizaje Continuo y Mejora Iterativa

Es importante aclarar una duda común de los equipos técnicos: **ChatGPT no aprende en tiempo real**. El modelo no cambia sus pesos matemáticos mientras chateas con él (eso sería inestable y peligroso).

El **Aprendizaje Continuo** en esta arquitectura se refiere a la capacidad del *sistema* (la organización y su infraestructura) para lanzar versiones cada vez más refinadas. La **Mejora Iterativa** asegura que cada versión sea un "escalón" más arriba en términos de seguridad, precisión y utilidad, basándose en la sabiduría colectiva de sus usuarios.

## Referencias

3. Adli, D. (s. f.). BPE vs WordPiece vs SentencePiece: A beginner-friendly guide to subword tokenization. Medium.  
<https://medium.com/@dhiyaadli/bpe-vs-wordpiece-vs-sentencepiece-a-beginner-friendly-guide-to-subword-tokenization-8047b39d82e0>
4. AIML. (s. f.). Explain the GPT architecture and compare GPT-style models.  
<https://aiml.com/explain-the-gpt-architecture-and-compare-gpt-style-models/>
5. Aman's AI Journal. (s. f.). Primers • Data filtering.

- <https://aman.ai/primers/ai/data-filtering/>
6. arXiv. (2023). Secrets of RLHF in large language models part I: PPO. <https://arxiv.org/html/2307.04964v1>
  7. arXiv. (2026). KV cache optimization strategies for scalable and efficient LLM inference. <https://arxiv.org/html/2603.20397v1>
  8. Artificial Intelligence Stack Exchange. (s. f.). What are the types of tokenizers?. <https://ai.stackexchange.com/questions/47214/what-are-the-types-of-tokenizers>
  9. Atlan. (s. f.). LLM training data lineage: Provenance, tracking & compliance. <https://atlan.com/know/training-data-lineage-for-llms/>
  10. AWS Security Blog. (s. f.). Securing the RAG ingestion pipeline: Filtering mechanisms. <https://aws.amazon.com/blogs/security/securing-the-rag-ingestion-pipeline-filtering-mechanisms/>
  11. Bansal, L. (s. f.). KV cache explained: The complete guide to KV cache in LLM .... Medium. <https://luv-bansal.medium.com/the-evolution-of-kv-cache-from-simple-buffers-to-distributed-memory-systems-df51cb8ce26f>
  12. Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... y Amodei, D. (2020). Language models are few-shot learners. arXiv. <https://arxiv.org/abs/2005.14165>
  13. Build Fast with AI. (2026). What is RLHF? The complete guide to training LLMs that actually work (2026). <https://www.buildfastwithai.com/blogs/what-is-rlhf-llm-training>
  14. Chen, Y. (s. f.). A rapid guide about LLM's training in parallelism. Medium. <https://medium.com/@yananchen1116/a-rapid-guide-about-llms-training-in-parallelism-d6edf0dba876>
  15. CircleCI. (s. f.). Automated version control for LLMs using DVC and CI/CD. <https://circleci.com/blog/automated-version-control-for-llms-using-dvc-and-ci-cd/>
  16. CodeSignal Learn. (s. f.). Comparing BPE, WordPiece, and SentencePiece in NLP. <https://codesignal.com/learn/courses/2-modern-tokenization-techniques-for-ai-llms/lessons/comparing-bpe-wordpiece-and-sentencepiece-in-nlp>
  17. Colossal-AI. (s. f.). Paradigms of parallelism. [https://colossalai.org/docs/concepts/paradigms\\_of\\_parallelism/](https://colossalai.org/docs/concepts/paradigms_of_parallelism/)
  18. Department of Electrical Engineering. (2026). Evaluations of LLMs. City University of Hong Kong. [https://www.ee.cityu.edu.hk/~lmpo/ee4016/pdf/2026\\_AI\\_L07C\\_LLM\\_Evaluations.pdf](https://www.ee.cityu.edu.hk/~lmpo/ee4016/pdf/2026_AI_L07C_LLM_Evaluations.pdf)
  19. Dipen. (s. f.). Tensor parallelism vs data parallelism: Which scales better?. Newline. <https://www.newline.co/@Dipen/tensor-parallelism-vs-data-parallelism-which-scales-better--127385a1>
  20. Emergent Mind. (s. f.). InstructGPT: Human-aligned language model. <https://www.emergentmind.com/topics/instructgpt>
  21. Emergent Mind. (s. f.). Scalable unlabeled data pipeline. <https://www.emergentmind.com/topics/scalable-unlabeled-data-collection-pipeline>
  22. Flagsmith. (s. f.). Rolling deployment vs. blue-green: Which strategy fits your

- pipeline?. <https://www.flagsmith.com/blog/rolling-deployment-vs-blue-green>
23. Future AGI Blog. (s. f.). Model vs data drift: How to identify and handle it. <https://futureagi.com/blog/model-vs-data-drift-how-to-identify-and-handle-it/>
  24. Galileo AI. (s. f.). Comparing model vs data drift and best detection practices. <https://galileo.ai/blog/model-vs-data-drift-detection>
  25. GeeksforGeeks. (s. f.). Introduction to Generative Pre-trained Transformer (GPT). <https://www.geeksforgeeks.org/artificial-intelligence/introduction-to-generative-pre-trained-transformer-gpt/>
  26. Hazra, S. (s. f.). InstructGPT deep dive: How RLHF turned raw language models into helpful assistants. Medium. <https://medium.com/@hazrasayan201442/instructgpt-deep-dive-how-rlhf-turned-raw-language-models-into-helpful-assistants-7b6da943e676>
  27. ITU iLibrary. (s. f.). Embedding drift: Detect, recompute, and budgeting. <https://itu-ilibrary.org/embedding-drift-detect-recompute-and-budgeting>
  28. Javeri, P. (s. f.). GPT Architecture. Decoder-only transformer architectures.... Medium. <https://medium.com/@prashunjaveri/gpt-architecture-0415e7a5796d>
  29. Kimutai, T. (s. f.). The ingestion-aware content strategy. Medium. <https://timkimutai.medium.com/the-ingestion-aware-content-strategy-9ece76603e0e>
  30. Klaus82. (s. f.). Blue-green and canary deployments: A deep dive into modern deployment strategies. Dev.to. <https://dev.to/klaus82/blue-green-and-canary-deployments-a-deep-dive-into-modern-deployment-strategies-48ci>
  31. LaunchDarkly. (s. f.). LLM evaluation: Tutorial & best practices. <https://launchdarkly.com/blog/llm-evaluation/>
  32. Leanware. (s. f.). LLM monitoring & drift detection guide | Metrics, tools & examples. <https://www.leanware.co/insights/llm-monitoring-drift-detection-guide>
  33. Meta-Intelligence. (s. f.). LLM fine-tuning data pipeline: Collection, annotation & synthetic data [2026]. <https://www.meta-intelligence.tech/en/insight-finetuning-data>
  34. Nebuly. (s. f.). LLM feedback loop. <https://www.nebuly.com/blog/llm-feedback-loop>
  35. NVIDIA Technical Blog. (s. f.). Mastering LLM techniques: Evaluation. <https://developer.nvidia.com/blog/mastering-llm-techniques-evaluation/>
  36. OpenAI. (2023). GPT-4 technical report. arXiv. <https://arxiv.org/abs/2303.08774>
  37. Orq.ai. (s. f.). LLM benchmarks explained: Significance, metrics & challenges. <https://orq.ai/blog/llm-benchmarks>
  38. Orq.ai. (s. f.). Understanding model drift and data drift in LLMs (2026 guide). <https://orq.ai/blog/model-vs-data-drift>
  39. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Lowe, A. J., Williams, W., ... y Ziegler, D. M. (2022). Training language models to follow instructions with human feedback. arXiv. <https://arxiv.org/abs/2203.02155>
  40. Pascal. (s. f.). 2024 Stanford LLM lecture analysis. Pascal's Substack. <https://p4sc4l.substack.com/p/2024-stanford-llm-lecture-analysis>
  41. Peri, P. (2026, 25 de marzo). LLM inference optimization: Quantization, KV cache, and serving at scale. <https://pr-peri.github.io/blogpost/2026/03/25/blogpost-llm-quantization-kv-cache.html>

42. ProjectPro. (s. f.). LLM evaluation benchmarks every AI engineer should know. <https://www.projectpro.io/article/llm-evaluation-benchmarks/1077>
43. Raschka, S. (s. f.). Machine learning Q and AI. <https://sebastianraschka.com/books/ml-q-and-ai-chapters/ch07/>
44. ResearchGate. (2026). Continuous feedback loops: Online fine-tuning of LLMs with user signals. [https://www.researchgate.net/publication/396233153\\_Continuous\\_feedback\\_loops\\_Online\\_fine-tuning\\_of\\_LLMs\\_with\\_user\\_signals](https://www.researchgate.net/publication/396233153_Continuous_feedback_loops_Online_fine-tuning_of_LLMs_with_user_signals)
45. Towards AI. (s. f.). LLM inference optimization: KV cache, batching, paged/flash attention. <https://pub.towardsai.net/llm-inference-optimization-730b2d718a44>
46. Towards Data Science. (s. f.). Meet GPT, the decoder-only transformer. <https://towardsdatascience.com/meet-gpt-the-decoder-only-transformer-12f4a7918b36/>
47. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., y Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
48. Wikipedia. (21 de abril de 2026). GPT-3. <https://en.wikipedia.org/wiki/GPT-3>
49. Yang, Z. (s. f.). How to build feedback loops for LLMs. Newline. <https://www.newline.co/@zaoyang/how-to-build-feedback-loops-for-llms--386075af>

## RESULTADO ESPERADO

El equipo debe ser capaz de:

- reconstruir el pipeline completo de ChatGPT
- entender cada componente a nivel técnico
- justificar decisiones arquitectónicas
- identificar cómo fluye la información en el sistema